PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

Yateen Vaviya (UCID 2023300251)
Vinayak Yadav (UCID 2023300264)
Sahil Gupta (UCID 2023300072)

Guide: Dr. Rupali Sawant Mentor: Prof. Dayanand Ambawade

Department of Computer Engineering, Sardar Patel Institute of Technology, Mumbai
Main Project Stage-II, End Semester Examination, November 2026

Introduction

- Idle computers are everywhere: college labs, friends' laptops, phones.
- Using them for a Python script normally needs three things first:
  - a configured cluster with a head node and matching software on every machine,
  - a program rewritten against a framework's API (Spark, Ray, Dask),
  - machines that can reach one another, which home and campus routers prevent.
- Remote execution over ssh avoids setup but gives no environment management, placement, isolation or fault tolerance.
- PipedPeer: **pipedpeer run script.py** runs the unchanged script on other machines, and spreads the parallel work inside it when that is faster.

Problem Statement

- Given an **unmodified Python script** and a small group's machines, possibly on **different networks behind NAT routers**, run the script and its parallel work on those machines:
  - with **no cluster to configure** and no head node or scheduler process,
  - with **no change to the script**,
  - reproducing the script's environment exactly on every machine,
  - joining machines across networks **without relaying** job data through a third machine,
  - never changing the result, and declining to distribute when it would be slower.

Objectives

- One binary per machine; joining takes one setup command and, across networks, one invite code.
- Build the script's environment as a pinned Nix closure; send a peer only the store paths it lacks.
- Substitute the parallel primitives the script already uses (Pool, joblib, pandas, NumPy, PyTorch training) at import time.
- Divide work by measured machine speed; keep work local when a cost model predicts no gain.
- Connect machines behind different routers directly: introducer, port mapping, hole punching; no relay.
- Train PyTorch models data-parallel across machines, with accuracy held within 1% of one device.
- Survive a worker, link or rank failing without changing the result; sandbox every job.
- Use an Android phone's CPU as a worker.

Literature Survey

System / paper	What it does	Limitation for our problem
Spark (Zaharia et al., 2010)	Cluster computing on resilient datasets	Configured cluster, Spark API, JVM
Ray (Moritz et al., OSDI 2018)	Tasks and actors; drop-in Pool and joblib	Head node, changed import, reachable machines
Dask (Rocklin, 2015)	Parallel NumPy/pandas, task scheduler	Scheduler process, changed code
Arjona et al., FGCS 2023 (Lithops)	Unmodified multiprocessing on serverless functions	Needs a cloud back end; no peers, no NAT
HTCondor / BOINC	Harvest idle machines for jobs	Central manager; project-specific
Nix (Dolstra et al., 2004)	Content-addressed, reproducible builds	Single machine; no peer-to-peer push
Ford et al., 2005; STUN/ICE	UDP hole punching across NATs	Building blocks, not an executor
DiLoCo (2023); local SGD (2020)	Train with infrequent synchronisation	Datacentre scale; need a framework

Research Gap

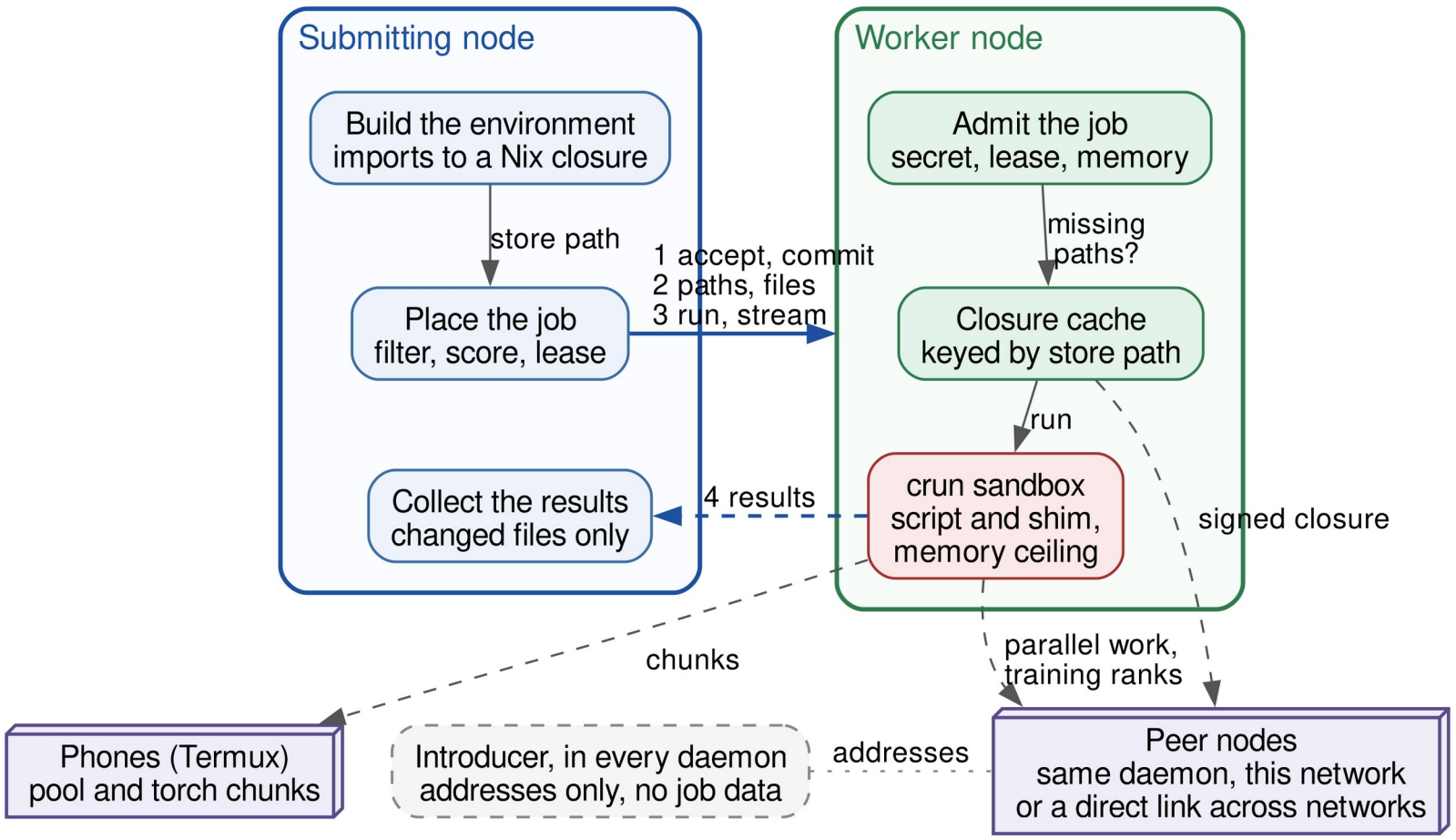
- Cluster frameworks remove the code change only partly and always need a configured cluster.
- Drop-in multiprocessing systems (Fiber, Lithops, Arjona et al.) need a provisioned back end: a cluster manager or a cloud account.
- None ships the script's exact environment to ordinary peer machines.
- None connects machines behind different home routers without a VPN or a relay.
- None decides per call whether distributing is worth it, and stays local when it is not.
- **Gap:** run unmodified Python across a small group's own machines, on any network, with no setup beyond one command, and never slower or different than running it locally by design.

Proposed System: PipedPeer

- Each machine runs **one daemon** from a single Go binary; any machine submits and works.
- **pipedpeer setup** installs everything; **pipedpeer invite / connect** joins another machine.
- **pipedpeer run script.py**:
  - scans imports and builds a pinned **Nix closure**,
  - places the job by live capacity under a lease,
  - sends only the missing store paths and the workspace,
  - runs it in a **crun** sandbox with a memory ceiling,
  - streams output back; changed files return.
- Inside the job, a **shim** spreads Pool, joblib, pandas, NumPy and torch work over the cluster.

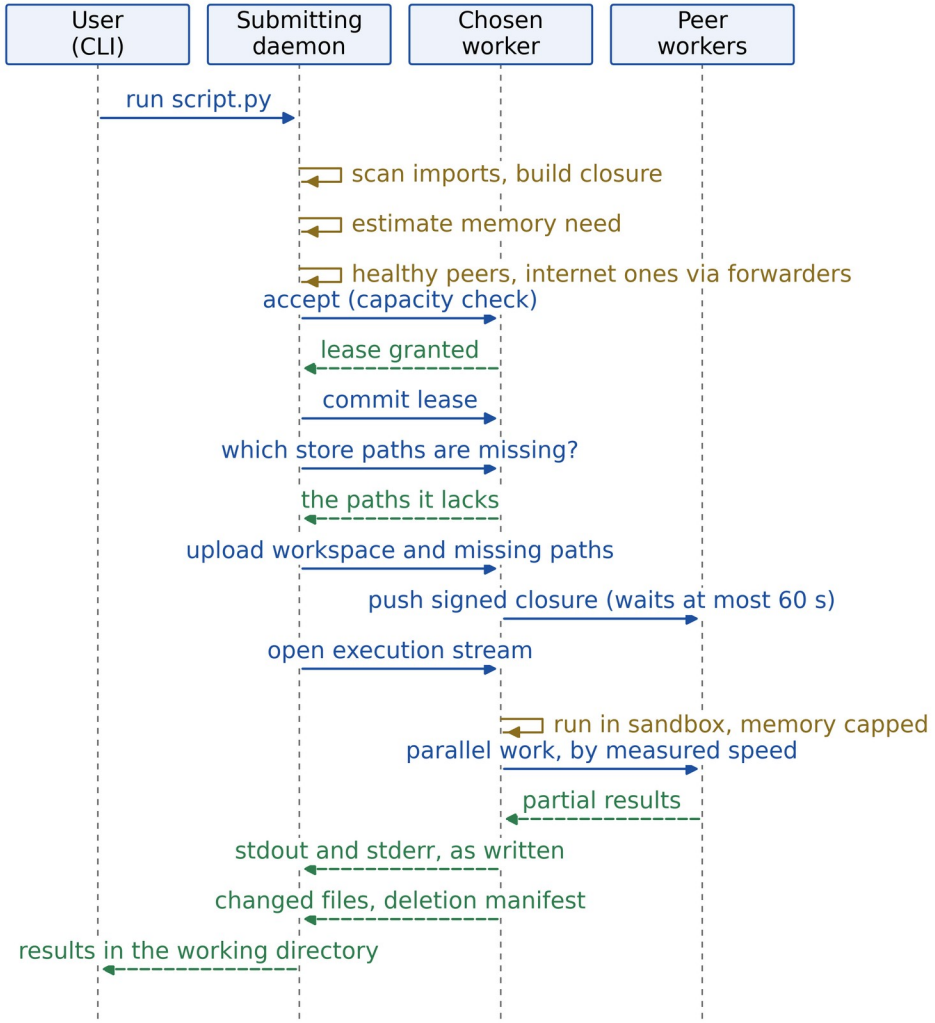
Command	What it does
setup	Install, open ports, start daemon
invite / connect	Join a machine with one code
run script.py	Run, spreading the work
status / dashboard	Machines and their work
doctor	Diagnose, with the fix
jobs / attach	History; reattach to a run
ui	Cluster web page
uninstall	Remove everything it added

System Architecture



- No head node: placement runs in the submitting client.
- Worker: admit, cache by store path, sandbox.
- Peers take parallel work and training ranks.
- Phones take chunks only.
- Introducer in every daemon: addresses only.

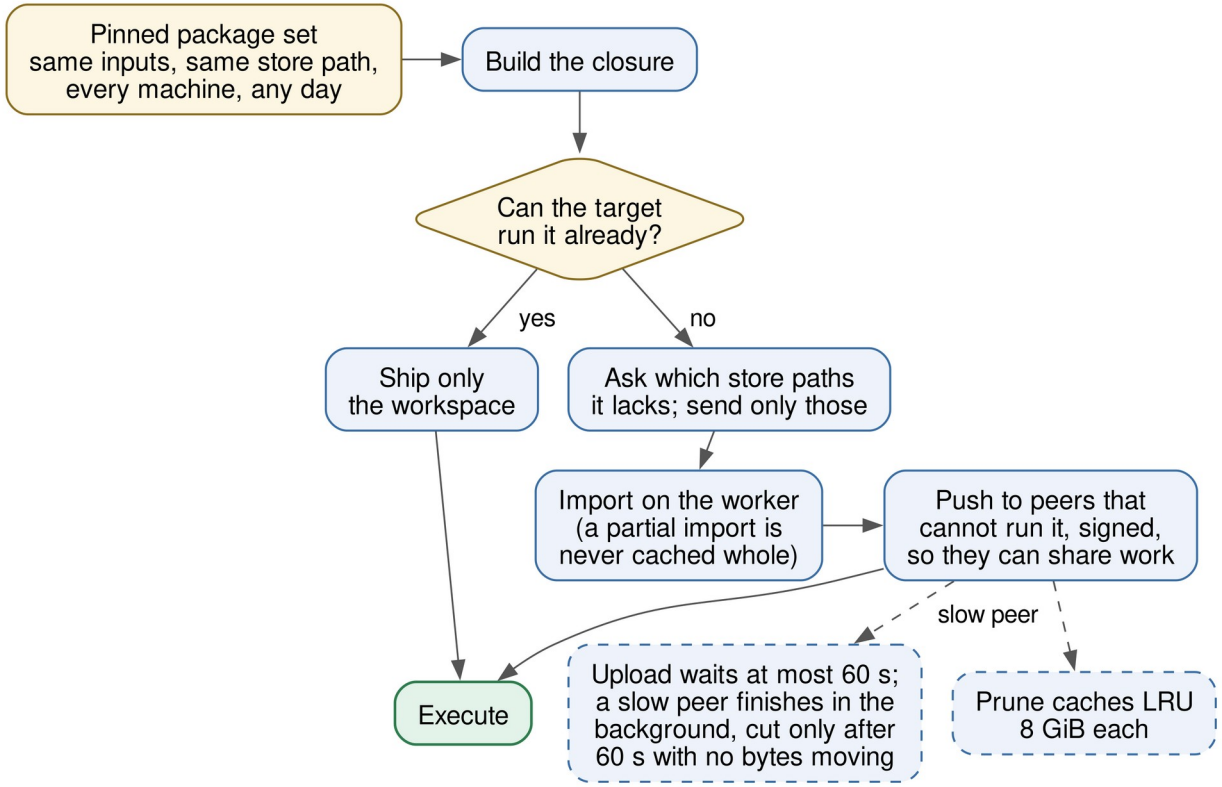
How a Job Runs

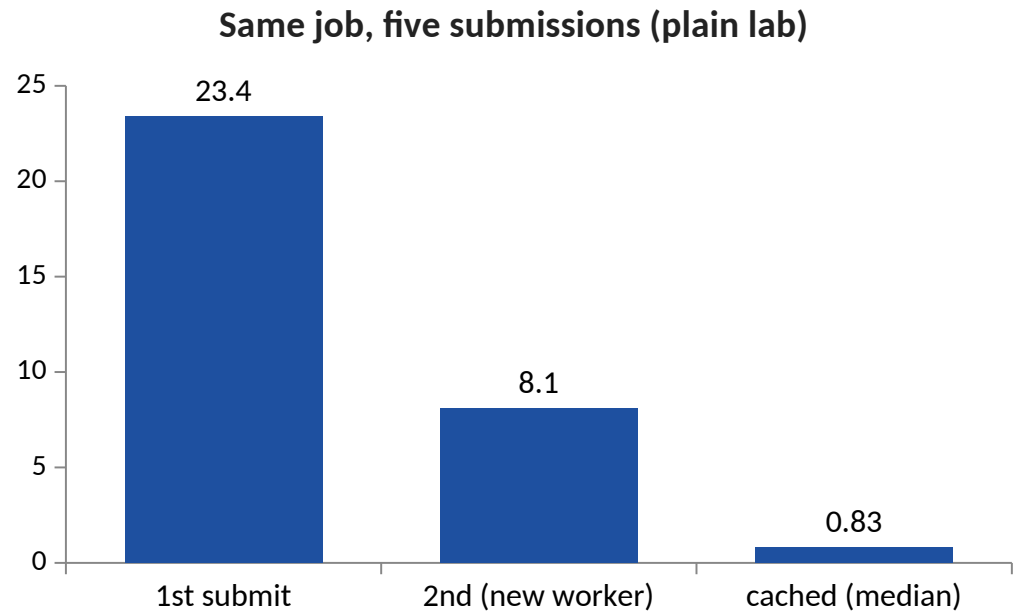


1. Scan imports; build or reuse the closure.
2. Filter and score candidates; three-phase lease (accept, commit, complete), renewed every 30 s.
3. Ask what the worker lacks; send only those store paths, then the workspace.
4. Run in a sandbox with the shim on the interpreter's path.
5. Parallel calls may go to peers that can run the same closure.
6. Stream output; return changed files; print where the time went.

- Ctrl+C stops the job on every machine; a dropped connection never kills it (reattach).

Environment Reproduction: Nix Closures

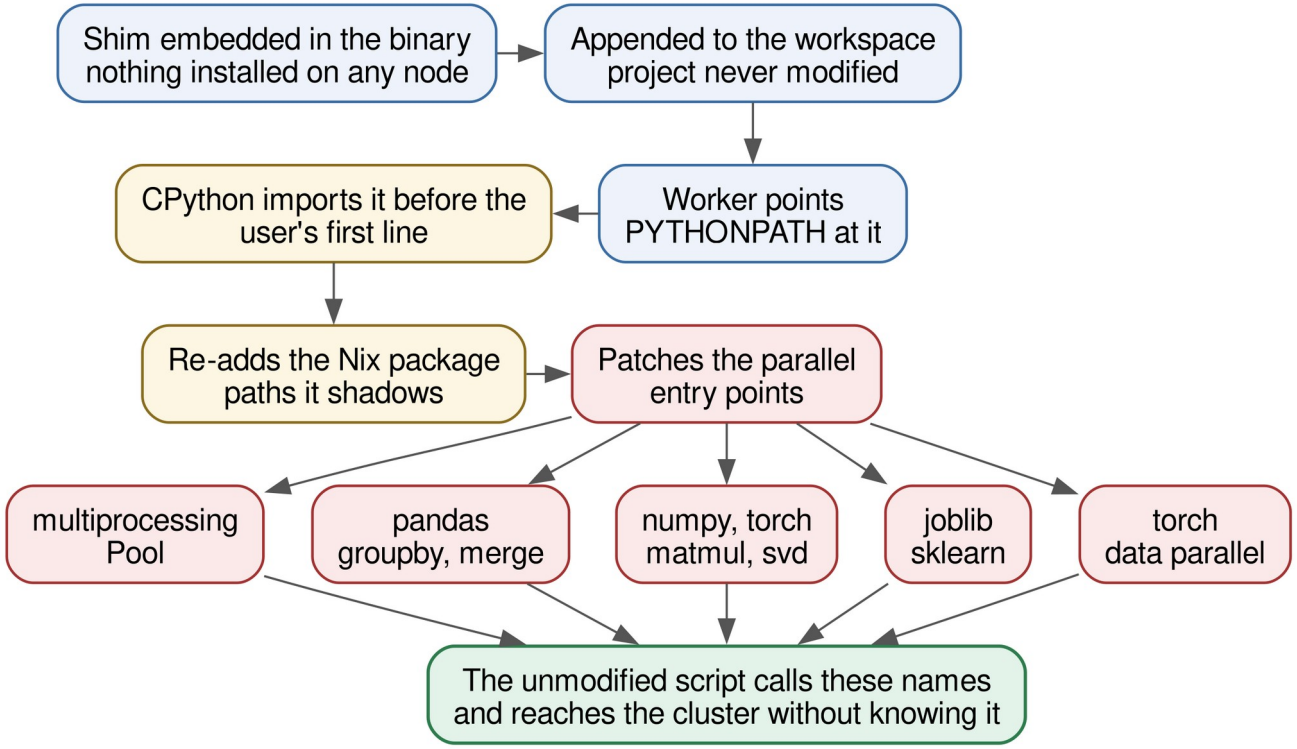




Closure diff (one package added)	Sent
Whole closure	206.0 MB
Only missing paths (7 of 43)	28.6 MB (-86%)

- Pinned package-set revision: same script, same store path, every machine and every day.
- Signed closures pushed peer to peer; only peers that can run it get distributed work.

Transparent Interception



- Imported before the script's first line; no line of the script changes.
- Patched on first import:
  - multiprocessing / concurrent.futures maps, joblib
  - pandas groupby, merge, chunked read_csv
  - NumPy products and decompositions
  - PyTorch data-parallel training
- Any error falls back to the original call.
- Startup **1.24×** plain Python (eager patching: 20×).
- A declined call costs about **0.12 s** per job.

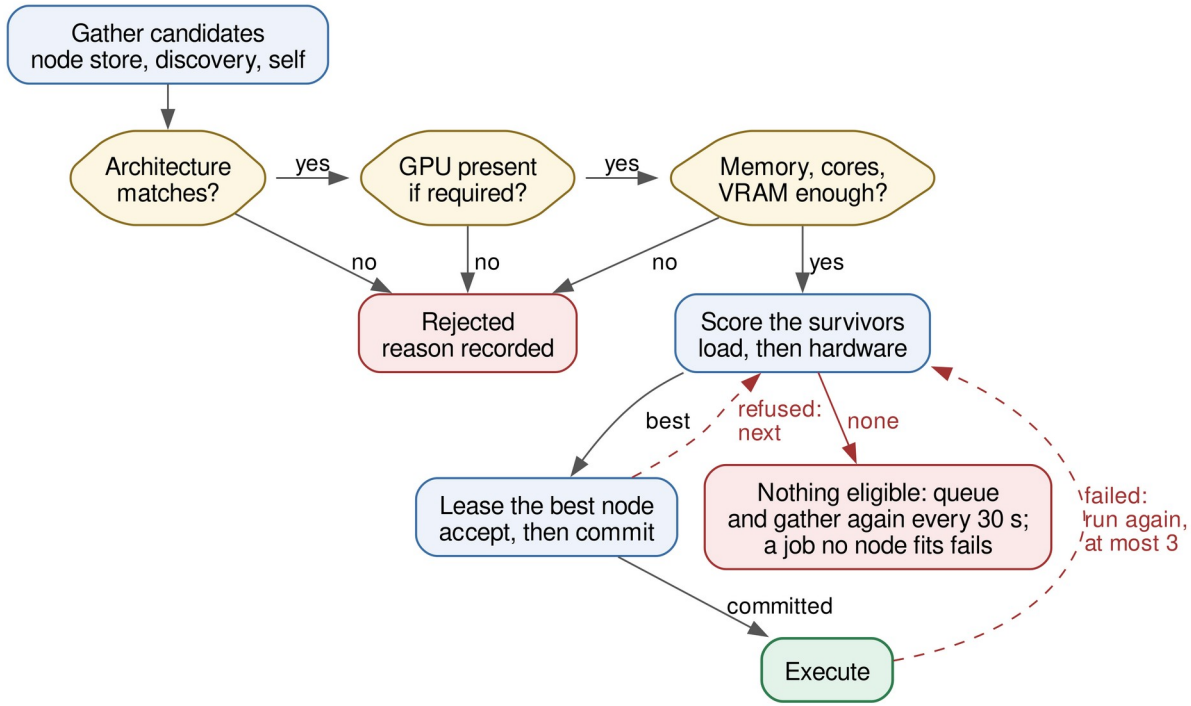
Dividing Work by Measured Speed

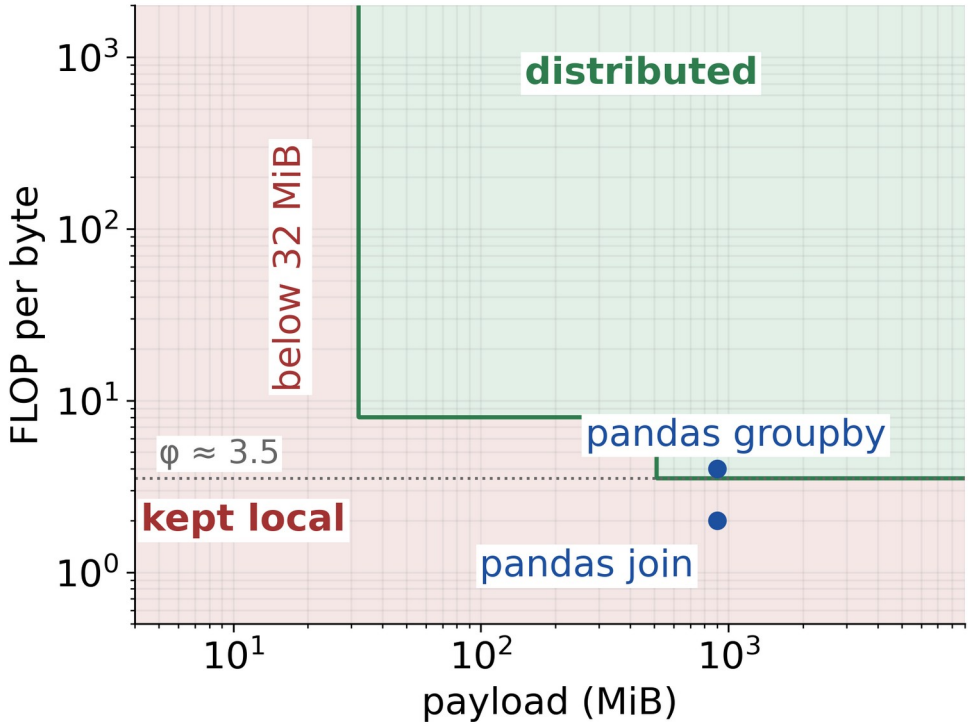
- Each device i: setup time o_i, effective rate r_i (measured, link cost folded in).
- Makespan T solves $\Sigma \max(0, T - o_i) \cdot r_i = W$ (W items).
- A device with $o_i \geq T$ cannot start in time and is dropped; repeat until the set settles.
- Rates are refitted from every part a device returns.
- Late parts are copied to a free device; first result wins (backup tasks).
- Idle local cores re-run unfinished parts, so a dead cluster degrades to a local pool.

Lab of 8, 2 and 1 CPUs	Time
Local only (derived)	≈ 2.3 s
Split by headcount	3.6 s (1.6× slower)
Split by measured speed	2.9 s (1-CPU node declined)

- Measured split beat headcount but was still ~1.3× slower than local at this size.
- A peer at 1/10 CPU: slowdown fell from 1.6× to 1.08× over four runs as rates converged.

Placement and the Cost Model

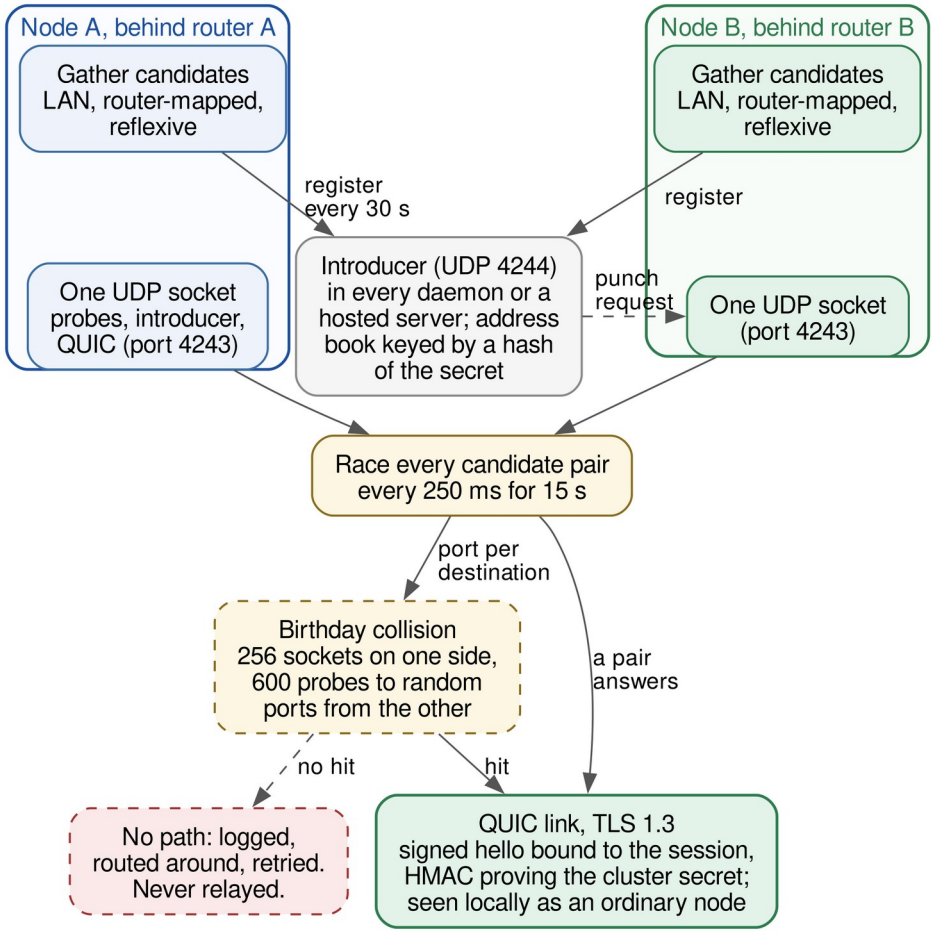




- Filters: architecture, GPU, free memory, cores, VRAM.
- Score: load terms subtract up to 1.0; hardware terms move it ±0.18.

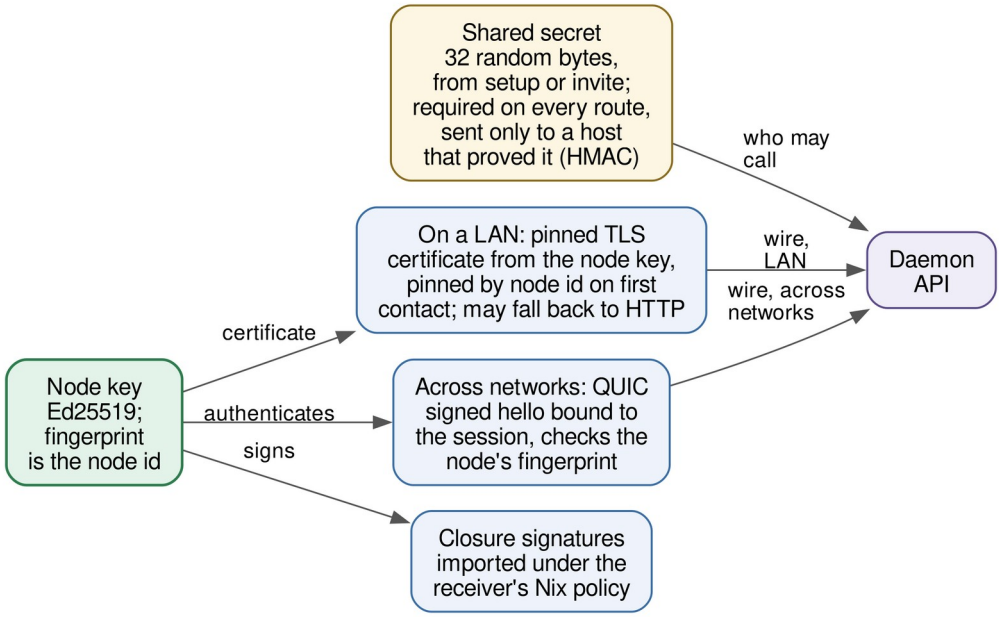
- Move work only if $\varphi/R > \rho/B + 1.3 \cdot \varphi/(R' K)$.
- Bandwidth measured only when offloading could win.

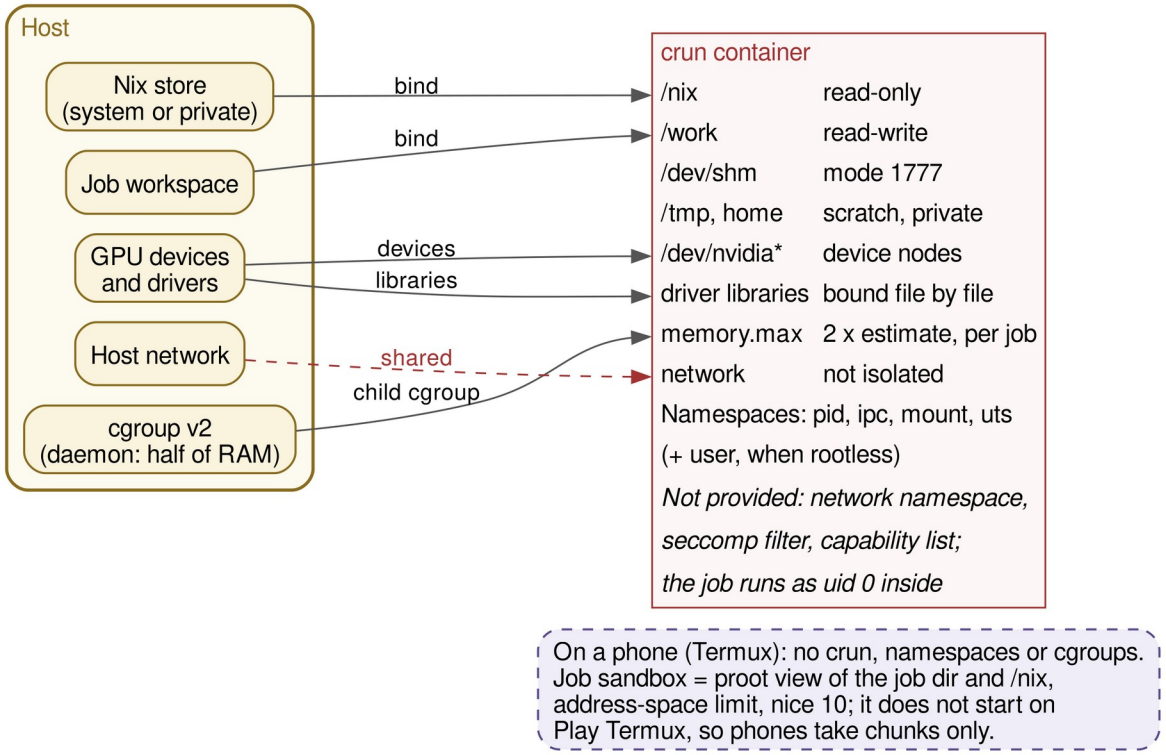
Joining Across Networks (NAT Traversal)



- Introducer in **every daemon**: address book + punch request; **no job data**.
- Candidates: LAN addresses, router port mapping (PCP, NAT-PMP, UPnP), reflexive address.
- Probes, introducer traffic and QUIC share **one UDP socket**; every pair probed every 250 ms for 15 s.
- Port-randomising NAT: 256 sockets vs 600 probes — **≈91% per round, computed**, not measured.
- QUIC link, hello signed with each node's **Ed25519** key + HMAC of the cluster secret.
- A node is its key, not its address: a machine that changes network rejoins on its own.
- No path? Reported with the reason and routed around — **never relayed**.

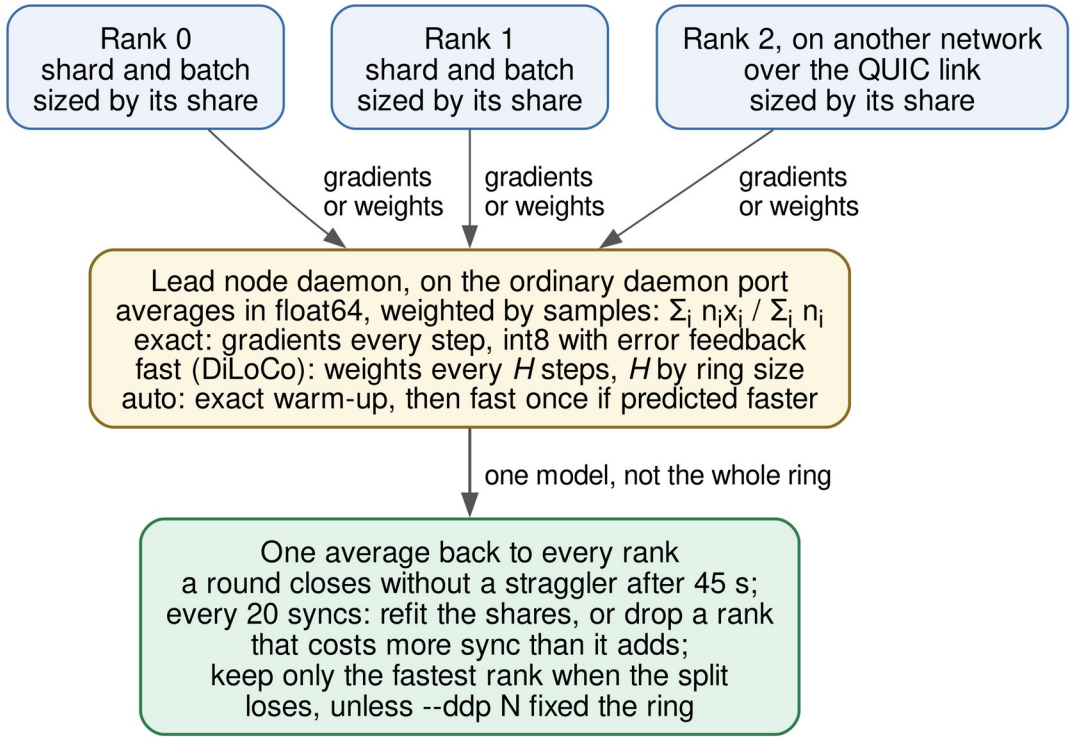
Trust and Sandboxing





- Closures signed by the sender; key checked before import.
- crun container per job: memory ceiling, GPU bound in.
- **Gaps:** TLS trust-on-first-use; one shared secret; signing keys not yet bound to identity.

Data-Parallel Training Through the Daemons



- Ranks post gradients to daemons — no rank-to-rank mesh needed.
- One **sample-weighted average**, sent as 8-bit with error feedback.
- Global batch split by **measured speed**; learning rate preserved.
- **Exact** sync every step, or **fast** sync (average weights every H steps).
- H chosen per ring size from accuracy runs: 32 (2 ranks), 16 (3), 4 (4); none for 5+.
- After warm-up, the ring is **released** if one machine alone is faster.
- A rank that dies: others take its share at the next epoch; a returning rank rejoins.

Phones as Workers

- Android phone joins from **Termux**, no root: **pipedpeer connect <code>**.
- Debian under proot-distro with single-user Nix; environment built for aarch64 from the public cache.
- Takes **Pool / joblib and torch chunks only** (chunks-only role).
- Whole jobs need a nested proot sandbox that Play-Store Termux cannot start.
- Sandbox is weaker than crun: proot view, memory limit, low priority.
- Keep it charging and run termux-wake-lock.

Samsung Galaxy S20 FE (Android 13)	Result
CPU script on 2 laptops + phone	phone took 8 of 192 items; same checksum
Environment built on the phone	27 s, from the cache
160 small torch models	phone trained 6; all converged
32 small torch models	phone took none
CPU script on the phone alone	62.4–66.0 s (Nix Python)

Fault Tolerance and Recovery

Drill	Result
Node's link cut during a 240-item map (two-site NAT testbed)	its 53 items re-ran on the submitter; checksum unchanged
A worker killed during a remote job	node reported gone 16 s after the kill
A training rank killed	remaining rank finished all 80 epochs (90.5 s)
A rank's machine back during training	rejoined at the next epoch, 4 s after its daemon started
Ctrl+C during a parallel map	peers stopped within 2 s
Submitter's network removed for 200 s	job ran once; result back ~15 s after reconnecting
Detach, then attach	followed live; replayed in 1.0 s once finished

Single points of failure remain: the submitter of a parallel script and the machine of a training ring's rank 0.

Implementation

- **Go 1.25**, one binary (client + daemon), **46 packages**.
- Embedded **Python shim**; links on **quic-go**; environments with **Nix flakes**; jobs in **crun**.
- Web page (pipedpeer ui) and terminal dashboard.
- Tests: **1,012 of 1,017 pass**, 4 skipped, 1 timing-sensitive failure (fresh run, 2 Nov 2026).
- **Mutation audit:** 95 features disabled one at a time — the tests caught **95 / 95**.
- CI: formatting, vet, race detector, skip check, never-3×-slower benchmarks.

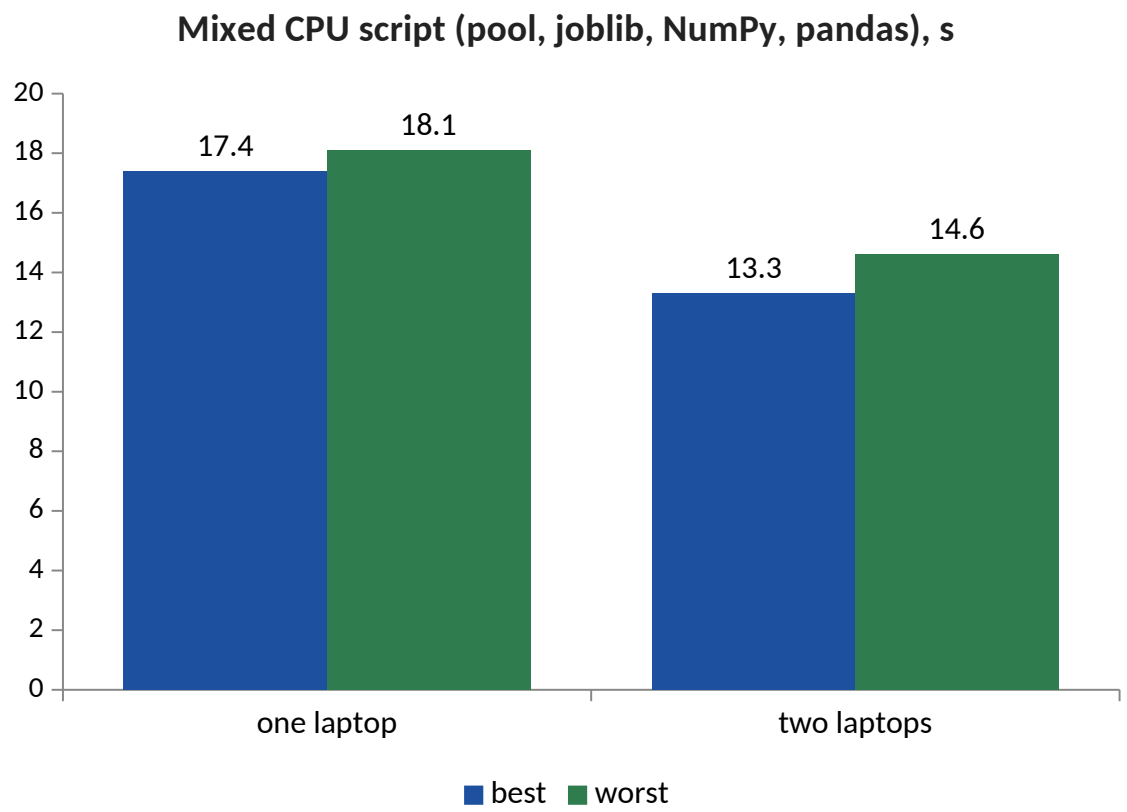
Area	Technology
Language	Go 1.25 + Python shim
Environments	Nix flakes (pinned nixpkgs)
Sandbox	crun (OCI), cgroup v2
Transport	QUIC (quic-go), TLS 1.3
Identity	Ed25519 node keys, HMAC
Training	PyTorch, 8-bit gradients
Phone	Termux + proot-distro

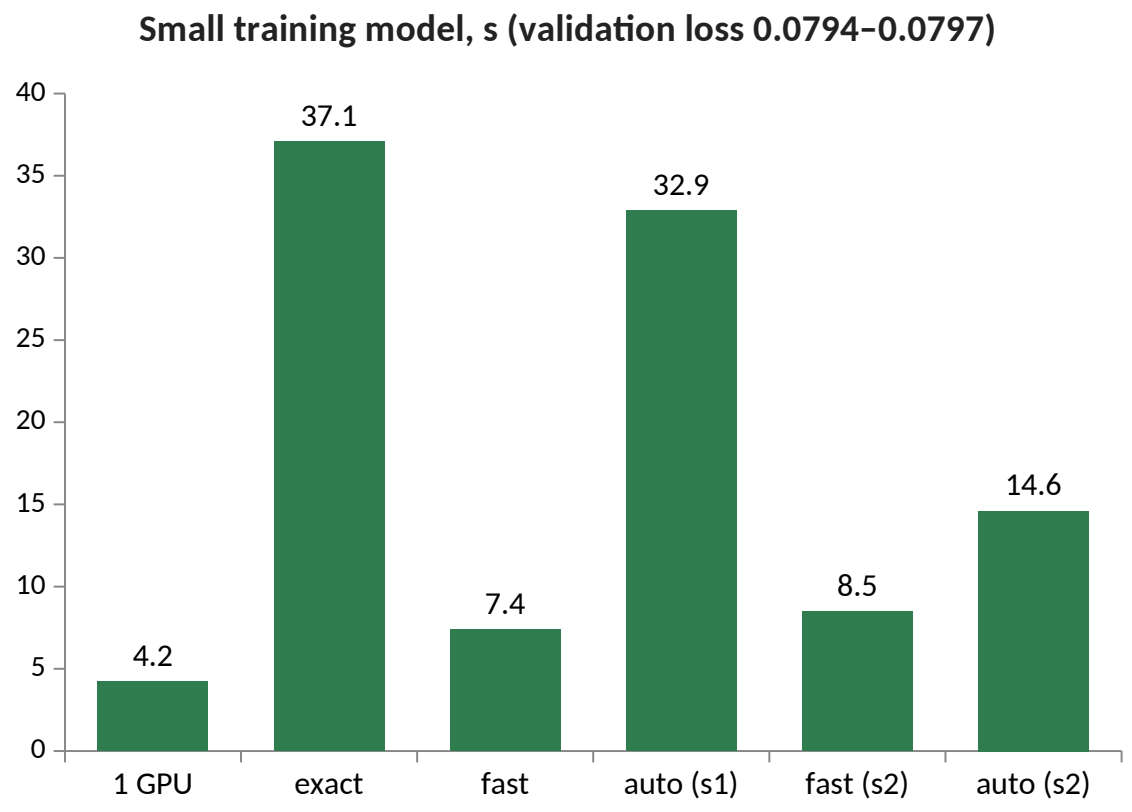
Experiment Setup

Setting	What it is	What it can show
Plain lab	3 worker containers sharing one laptop	Correctness, overhead, caching — not speedup
Quota lab (emulated)	Fixed CPU/memory per container; LAN / Wi-Fi / 5G shaped with netem	How cost grows; emulated speedup
Two-site testbed	Two private networks behind NAT router containers + introducer	Joining across NATs; failover
Physical set A	Ryzen 7 + RTX 3060 and Core i5 + RTX 3050 in two homes; Galaxy S20 FE	Real distribution (over Tailscale)
Physical set B	20-core Core i9 + Ryzen laptop; cloud host as introducer	Real pool map; introducer traffic

- Lab times: medians of 3–5 runs. Training: one seed (accuracy: mean of 3–6 seeds).
- Physical runs: single runs or ranges over 2–3 runs — evidence, not a controlled scaling study.

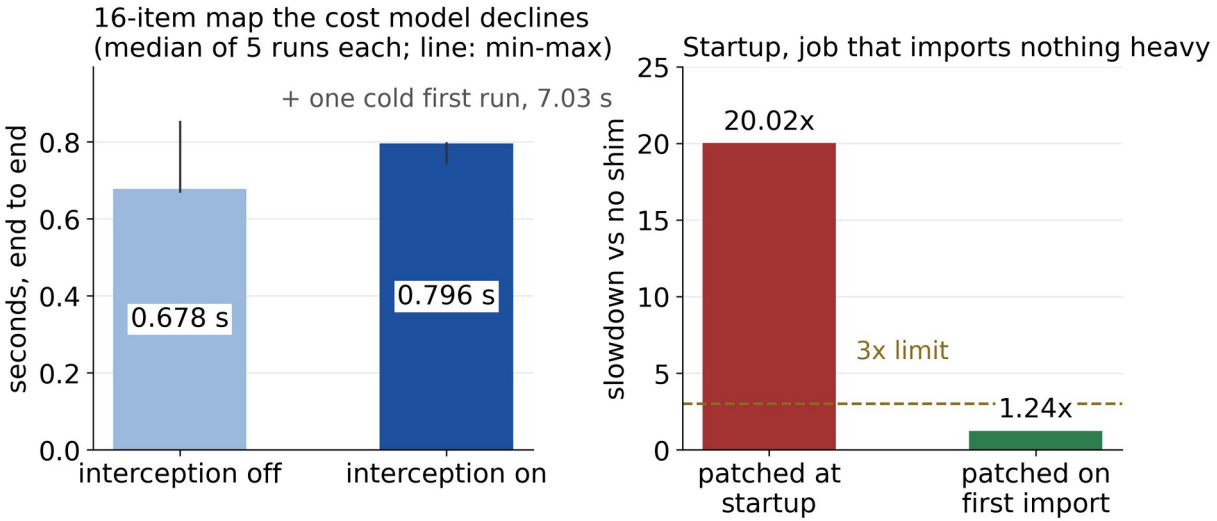
Results: Two Laptops in Different Homes





- CPU work gained on two machines; the small model was fastest on one GPU — splitting it did not pay.
- Fast sync 5.0× faster than exact at about the same loss; s1/s2 = two sessions on different builds.

Results: Overheads and Other Measurements

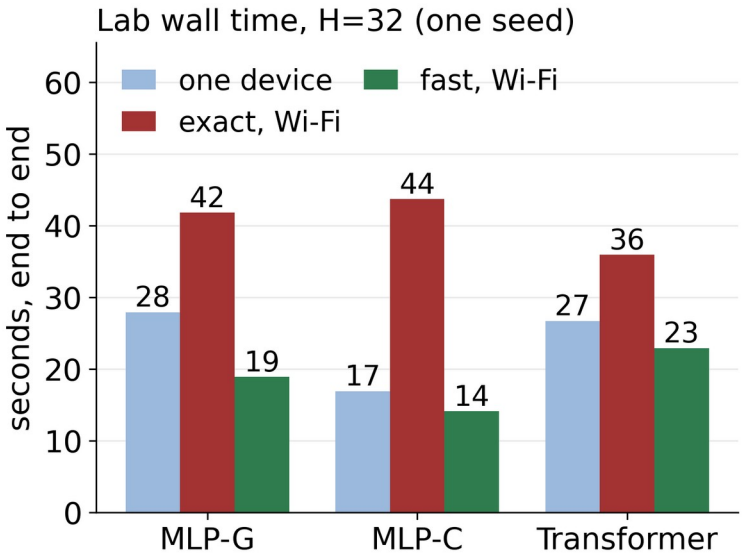


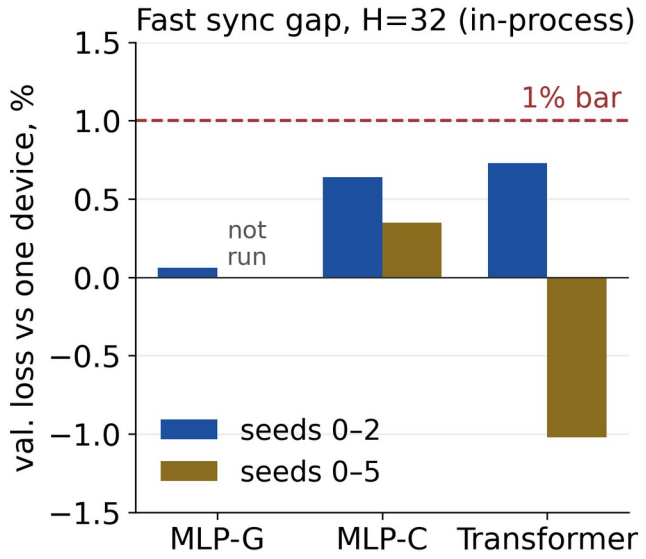
Emulated nodes (2 CPU each)	1	2	3	4
Wall time, s (loaded host)	2.37	1.15	0.95	0.81

Emulated quotas on one host: shows growth, not a physical-machine speedup. No CPU step grew faster than its expected order.

Measurement	Result
Startup with interception	1.24× plain Python
Declined map (16 items)	+0.12 s per job
crun vs bubblewrap start	+15–21 ms
Pool map, Core i9 alone vs + peer	5.8–6.4 vs 4.9–5.4 s
Training over 54 Mbit/s: 1 machine / 2 ranks fp16 / int8	54.9 / 69.4 / 56.1 s
Introducer traffic vs peer traffic	0.2 MB vs ~30 MB

Results: Training Accuracy and Bytes





Model (H = 32, 2 ranks)	Loss gap vs one device	MiB per rank, exact / fast	Lab time LAN: exact / fast / one device
MLP-G (559k params)	+0.06%	81.1 / 10.7	20.0 / 15.2 / 27.9 s
MLP-C (559k params)	+0.64%	98.9 / 10.7	14.6 / 10.4 / 16.9 s
Transformer (70k params)	+0.73%	21.7 / 2.4	22.7 / 19.1 / 26.7 s

Loss gap: mean of seeds 0–2, in-process. Lab times: quota lab, one seed, equal 2-CPU containers.

Comparison with Existing Systems

Property	PipedPeer	Spark	Ray	Dask	Lithops	ssh
Runs unmodified Python	yes	no	partial	partial	partial	yes
No cluster or back end to configure	yes	no	no	no	no	partial
No head or master node	yes	no	no	no	n/a	yes
Reproduces the environment	yes	partial	partial	partial	partial	no
Joins NATed machines, no relay	yes	no	no	no	n/a	no
Survives losing a node	partial	yes	yes	yes	yes	no
Authenticated, encrypted	partial	yes	partial	partial	yes	yes
Runtimes beyond Python	no	yes	partial	no	no	yes
Proven in production	no	yes	yes	yes	yes	yes

Qualitative, from published designs — no wall-time benchmark against Ray, Dask or Lithops was run.

Limitations

- Physical evaluation covers two or three machines and one phone, mostly single runs — not a controlled scaling study.
- No wall-time comparison with Ray, Dask or torchrun.
- Two homes without a reachable introducer need an overlay (e.g. Tailscale); there is no relay by design.
- Single points of failure: the submitter of a parallel script, and rank 0's machine.
- Python on Linux only; NVIDIA GPUs in practice. Phones take chunks only.
- Security: TLS trust-on-first-use, one shared secret, signing keys not bound to identity.
- Fast sync validated for 2–4 ranks; the 1% bar holds for the mean over seeds, not every seed.
- Cost-model and placement constants are hand-set, not calibrated across hardware.

Live Demo

```
$ pipedpeer connect <code>        # join with one line
$ pipedpeer status
$ python3 cpu.py                  # baseline TOTAL
$ pipedpeer run cpu.py            # same results, spread
$ pipedpeer run gpu.py            # one GPU or a split?
$ pipedpeer run --ddp 2 --ddp-sync fast gpu.py
$ pipedpeer run both.py           # CPU phase, then GPU
$ pipedpeer run --remote gpu.py   # whole job elsewhere
$ pipedpeer ui                    # cluster web page
$ pipedpeer run cpu.py --items 600   # cut Wi-Fi mid-run
```

- Join: ✓ **connected: N other machine(s)**.
- cpu.py: same checksum lines, lower TOTAL, and **ran on <machine> 60%, <machine> 40%.**
- gpu.py: keeps one GPU when the split would be slower — and says why.
- Phone appears in the **ran on** line for CPU work.
- Wi-Fi off mid-run: same checksum; work re-run elsewhere; machine returns by itself.

Conclusion and Future Scope

Conclusion

- Unmodified Python runs across a small group's machines with one command, no cluster and no relay.
- Environments are shipped as closures diffed by store path (0.83 s warm vs 23.4 s cold).
- CPU-bound work gained on two machines; small GPU models stayed on one GPU — the system measures and declines.
- Fast sync kept accuracy within 1% at 7.6–9.2× fewer bytes.

Future scope

- Bind signing keys to identities; require TLS; per-user authorisation.
- Survive the loss of the submitter and of rank 0.
- Whole jobs on phones; AMD GPUs.
- Locality- and topology-aware placement.
- Controlled, repeated evaluation on more machines against Ray, Dask and Lithops.

Execution Plan, Budget and Distribution of Work

Member	Responsibilities
Yateen Vaviya	[fill in before the exam]
Vinayak Yadav	[fill in before the exam]
Sahil Gupta	[fill in before the exam]

- **Timeline:** 12 April – November 2026 (first to latest commit); each phase on its own branch, merged into dev after review.
- **Budget:** no hardware bought — open-source software, our own laptops and phone. [confirm: any cost of the cloud host used as introducer]

Research Publication

- **"PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination"**
  - Y. Vaviya, V. Yadav, S. Gupta, R. Sawant, D. Ambawade — IEEE conference format, 6 pages.
  - Status: [fill in — e.g. submitted to <conference>, <date>]; plagiarism report attached.
- Journal version prepared for the **Journal of Parallel and Distributed Computing** (Elsevier).
- Every number in the paper and report traces to a recorded measurement in the repository.

References

M. Zaharia et al., Spark: Cluster computing with working sets, HotCloud, 2010.

P. Moritz et al., Ray: A distributed framework for emerging AI applications, OSDI, 2018.

M. Rocklin, Dask: Parallel computation with blocked algorithms and task scheduling, SciPy, 2015.

A. Arjona, G. Finol, P. García-López, Transparent serverless execution of Python multiprocessing applications, Future Gener. Comput. Syst. 140 (2023) 436–449.

E. Dolstra, M. de Jonge, E. Visser, Nix: A safe and policy-free system for software deployment, LISA, 2004.

B. Ford, P. Srisuresh, D. Kegel, Peer-to-peer communication across network address translators, USENIX ATC, 2005.

J. Iyengar, M. Thomson, QUIC: A UDP-based multiplexed and secure transport, RFC 9000, 2021.

S. Li et al., PyTorch distributed: Experiences on accelerating data parallel training, PVLDB 13(12), 2020.

A. Douillard et al., DiLoCo: Distributed low-communication training of language models, arXiv:2311.08105, 2023.

T. Lin et al., Don't use large mini-batches, use local SGD, ICLR, 2020.



Thank You

Questions?

Yateen Vaviya, Vinayak Yadav, Sahil Gupta

Guide: Dr. Rupali Sawant · Mentor: Prof. Dayanand Ambawade